

浙江大学 实验报告

课程名称: 微机原理及应用
实验名称: 实验三 三键多信息输入实验
指导老师: Kimi Moonshot, Poe
实验类型: 仿真验证实验

专业: 生物医学工程
姓名: 李田所
学号: 322114514
日期: 1919 年 8 月 10 日
地点: 下北沢

目录

1. 实验目的和要求	2
1.1. 实验要求	2
1.2. 实验目的	2
2. 实验内容和原理	2
3. 主要仪器设备	2
4. 操作方法和实验步骤	2
4.1. 需求分析	2
4.2. 系统设计	2
4.3. 硬件设计	3
4.3.1. 输入	3
4.3.2. 输出	3
4.3.3. 电路	3
4.4. 软件设计	4
4.4.1. 硬件资源规划	4
4.4.2. 流程图	5
4.4.3. 代码	5
5. 实验结果和分析	9
5.1. 系统测试	9
5.2. 分析	13
6. 思考和讨论	14
6.1. 扩展思考题	14
6.2. DEBUG	14
6.3. 总结	15
6.4. 参考文档的思考	15

1. 实验目的和要求

1.1. 实验要求

1. 在一些工业与生活应用场景中，受条件所限，只能提供有限个输入按键，但需要输入多个控制参数及参数的大小，比如洗衣机、电饭煲等家用电器的控制面板等，某些液晶显示屏参数调节面板等。本实验的任务是设计基于叁键输入的4参数信息输入应用系统，各参数值可做单位增一操作；用4个LED灯分别标示4个参数，但仅当前执行输入的活动参数对应的LED灯亮，其它参数对应的灯均熄灭；参数的数值范围为0—8，用LED亮灯个数显示，最大8个灯全亮，最小全暗。
2. 要求实验前，对实验任务进行认真分析，写出需求分析报告和系统设计报告。
3. 要求写硬件设计报告，绘出硬件电路图。
4. 进行软件设计，画出软件流程图。
5. 写出系统测试方案，在按方案进行测试后，写出测试分析报告。
6. 写出实验体会。

1.2. 实验目的

1. 掌握和熟悉循环与多分支汇编程序设计技巧。
2. 掌握和熟悉 I/O 端口的复杂信息输入与输出方法。
3. 通过本实验三键多参数设置方法进行体验，充分掌握这些控制方法的基本应用操作技巧。

2. 实验内容和原理

在一些简单而实用的应用设备中，往往采用双键或三键进行信息输入，完成多参数的设置。本实验的内容是设计三键信息输入系统，实现多参数的输入控制。

对多参数输入，可用一个按键进行参数的选择，每按一次，更改一次参数，循环选择；当按另一个按键时，对参数进行增量操作，直到最大值后，保持在最大值；

第三只按键每按一次执行减量操作，直到最小值，并保持在最小值。

3. 主要仪器设备

计算机、Proteus 8.9 仿真软件

4. 操作方法和实验步骤

4.1. 需求分析

- 参数选择与显示：按下参数按钮，参数切换，4个LED代表4个参数，每次只亮一个LED，4个循环选择。
- 参数数值增加/减少与显示：按下参数数值增加/减少键，对应参数的数值增加/减少。数值为0~8，分别用0~8个LED点亮来对应表示。
- 参数与数值的对应存储：某参数数值被设置后，循环切换回该参数，显示的数值仍为切换参数前的数值。

4.2. 系统设计

- 实验装置以单片机为核心
- 输入控制：3 个输入端口，分别检测参数选择、数值增加、数值减少。
- 输出控制：12 个输出端口，各驱动一个 LED。其中 4 个端口分别指示一个参数，8 个端口组成一个 9 级参数显示条，显示参数的数值。
- 控制程序：检测并确定当前活动参数，点亮对应的参数灯；若检测到参数数值增减按钮按下，则执行增减，并显示数值。所以，控制程序需要循环检测三个按键的状态，若发现按键从断开到闭合的变化，则执行操作。

4.3. 硬件设计

系统核心采用 8051 单片机，该单片机在不进行外扩数据存储器 and 程序存储器的条件下，共有 32 个 I/O 可供使用。功能强，应用面广，价格低。

用 Proteus 新建一个工程，绘出 8051 单片机应用系统电路，在 P1.0、P1.1、P1.2 三个端口上分别连接一个开关，P2 口上连接 8 个 LED 灯，P3.4~P3.0 连接 4 个 LED 灯。普通 LED 灯可由 I/O 端口直接驱动，但当采用大功率 LED 灯时 i/o 端口驱动电流不足，可以采用三极管驱动。

4.3.1. 输入

P1 三个端口连接 3 个按钮，低电平有效。

按钮没有被按下时断开，对应端口高电平；按下时闭合，对应端口低电平。

选择参数 SEL	参数数值增加 INC	参数数值减少 DEC
P1.2	P1.1	P1.0

4.3.2. 输出

4.3.2.1. 参数选择

P3 的低四位连接 4 个 LED，低电平有效。

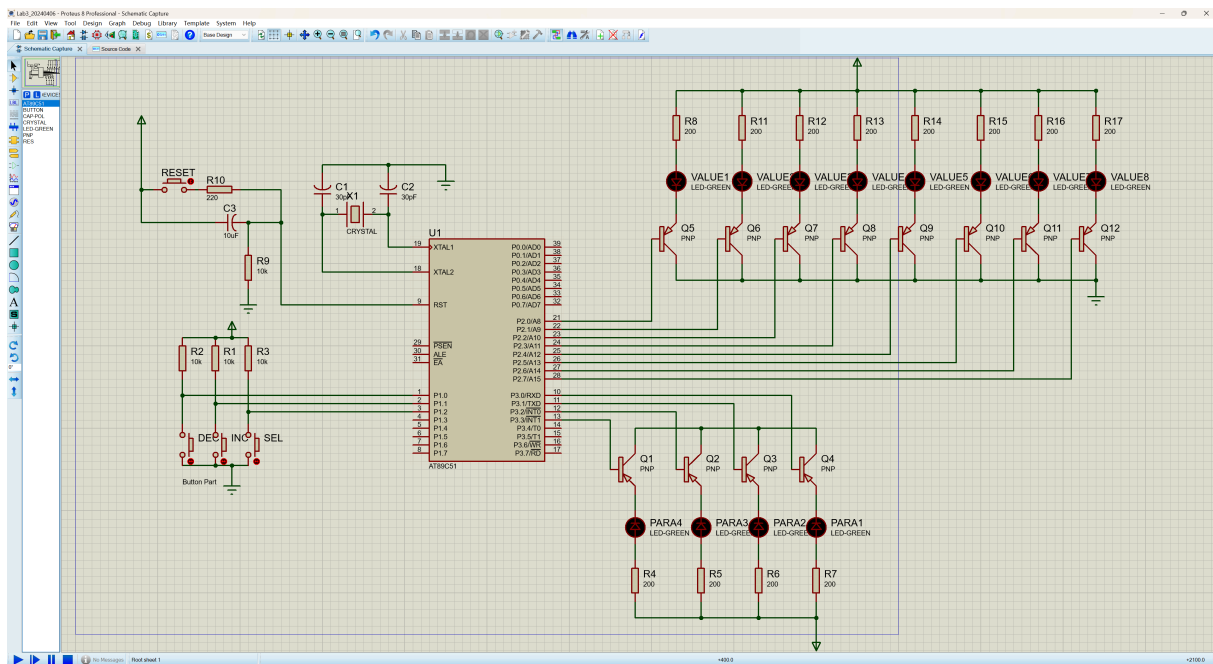
每个端口对应一个参数，亮灯表示当前操作的参数，故四个端口灯，同时有且仅有一个灯亮。

4.3.2.2. 参数数值

P2 的 8 个 IO 口连接 8 个 LED，低电平有效。显示活动参数的数值。

参数数值共 9 级，从 0 到 8，对应灯从左到右亮 0 到 8 个。

4.3.3. 电路



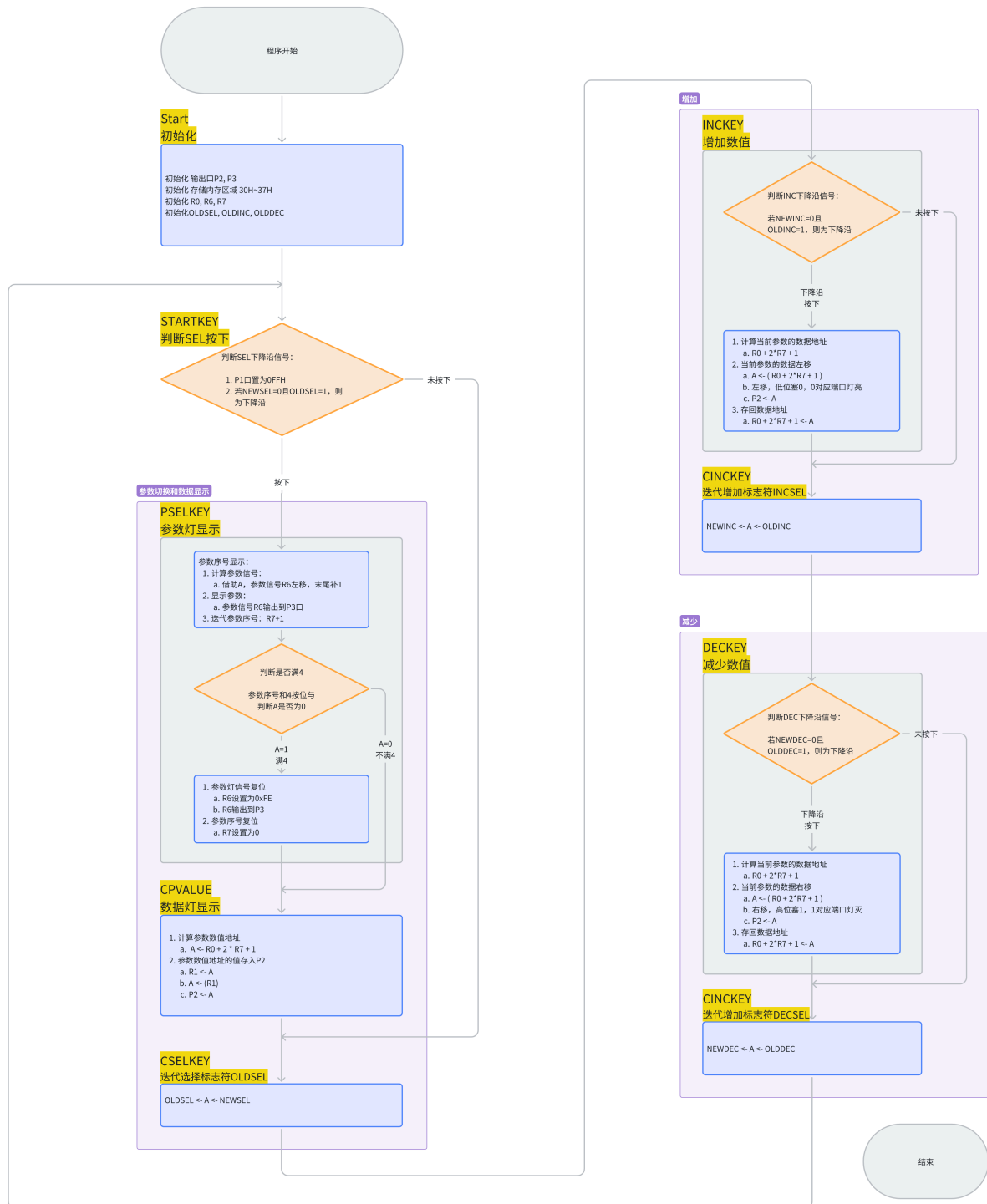
4.4. 软件设计

4.4.1. 硬件资源规划

- R0
 - 参数资源基地址
 - 始终为 0x30H
 - $R0 + 2 * R7$:
 - $R0 + 2 * R7 + 1$: 参数数值
- R1
 - 临时寄存器
- R6
 - 参数状态 输出信号
 - 0b11111110 初始值
 - 低电平有效, 0 对应端口的 LED 亮
- R7
 - 参数状态 序号
 - 0x00H 初始值
 - 范围[0, 1, 2, 3], 大于等于 4 时置零 (实际程序中由于 R7 循环增加, 所以只需判断其是否等于 4)
 - 每按一次 SEL, $R7+1$
- P2
 - 参数数值 显示
- P3

▸ 参数状态 显示

4.4.2. 流程图



4.4.3. 代码

```

; =====
; 定义
; =====
OLDSEL BIT 02H ; 参数选择键原状态

```

```

        OLDINC  BIT    01H    ; 参数增 1 键原状态
        OLDDEC  BIT    00H    ; 参数减 1 键原状态

        NEWSEL  BIT    P1.2   ; 新读入参数选择键状态
        NEWINC  BIT    P1.1   ; 新读入参数增 1 键状态
        NEWDEC  BIT    P1.0   ; 新读入参数减 1 键状态

;=====
; 变量
;=====

;=====
; 复位和中断向量
;=====

; 复位向量
org    0000h
jmp    Start

;=====
; 代码段
;=====
        org    0100h

Start:
        MOV    P2, #0FFH    ; 初始化 P2 为 0xFF
        MOV    P3, #0FEH    ; 初始化 P3 为 0xFE
        MOV    20H, #0FFH    ; 将初始值设置为 1

        MOV    30H, 00      ; 初始化 30H 为 0
        MOV    31H, #0FFH    ; 初始化 31H 为 0xFF

        MOV    32H, 00      ; 初始化 32H 为 0
        MOV    33H, #0FFH    ; 初始化 33H 为 0xFF

        MOV    34H, 00      ; 初始化 34H 为 0
        MOV    35H, #0FFH    ; 初始化 35H 为 0xFF

        MOV    36H, 00      ; 初始化 36H 为 0
        MOV    37H, #0FFH    ; 初始化 37H 为 0xFF

        MOV    R0, #030H    ; 初始化 R0 为 0x30

        SETB   OLDSEL       ; 将 OLDSEL 位设置为 1
        SETB   OLDINC       ; 将 OLDINC 位设置为 1
        SETB   OLDDEC       ; 将 OLDDEC 位设置为 1

        MOV    R6, #0FEH    ; 初始化 R6 为 0xFE
        MOV    R7, #00H     ; 初始化 R7 为 0

```

STARTKEY:

```
MOV P1, #0FFH      ; 将 P1 设置为 0xFF, 以读取信号
JB  NEWSEL, CSELKEY ; 如果 NEWSEL 位被设置, 则跳转到 CSELKEY
JNB OLDSEL, CSELKEY ; 如果 OLDSEL 位未被设置, 则跳转到 CSELKEY
```

; SEL 下降沿, 被按下

PSELKEY: ; 首先检测参数选择键

```
; CLR OLDSEL      ; 清除 OLDSEL 位
```

```
SETB C            ; 设置进位标志
```

```
MOV A, R6          ; 将 R6 移动到累加器
```

```
RLC A              ; 通过进位左旋转
```

```
MOV P3, A           ; 将累加器移动到 P3
```

```
MOV R6, A           ; 将累加器移动到 R6
```

```
INC R7              ; R7 加 1
```

```
MOV A, R7            ; 将 R7 移动到累加器
```

```
ANL A, #04H         ; 与 0x04 进行按位与
```

```
JZ  CPVALUE         ; 如果结果为零, 即参数序号 R7 未超过 3, 则跳转到 CPVALUE
```

显示数值

; 否则参数序号 R7 超过 3, 重置参数序号 R7

```
MOV R6, #0FEH       ; 将 R6 设置为 0xFE
```

```
MOV P3, R6           ; 将 R6 移动到 P3
```

```
MOV R7, #00H         ; 将 R7 设置为 0
```

CPVALUE: ; 显示参数数值

```
MOV A, R0            ; 将 R0 移动到累加器
```

```
ADD A, R7             ; 将 R7 加到累加器
```

```
ADD A, R7             ; 将 R7 加到累加器
```

```
INC A                 ; 累加器加 1
```

```
MOV R1, A             ; 将累加器移动到 R1
```

```
MOV A, @R1            ; 将地址 R1 处的值移动到累加器
```

```
MOV P2, A             ; 将累加器移动到 P2
```

CSELKEY: ; 处理参数选择键的标志符

```
MOV C, NEWSEL        ; 将 NEWSEL 位移动到进位标志
```

```
MOV OLDSEL, C         ; 将进位标志移动到 OLDSEL 位
```

; 然后检测参数数值增加键

INCKEY:

```
JB  NEWINC, CINCKEY ; 如果 NEWINC 位被设置, 则跳转到 CINCKEY
```

```
JNB OLDINC, CINCKEY ; 如果 OLDINC 位未被设置, 则跳转到 CINCKEY
```

```
MOV A, R0
```

```
ADD A, R7
```

```
ADD A, R7
```

```
MOV R1, A
```

```
; MOV A, @R1      ; 将地址 R0+2*R7 处的值移动到累加器
```

```

; INC A          ; 累加器加 1
; ANL A, #08H    ; 与 0x08 进行按位与
; JNZ CINCKEY    ; 如果结果为 1 (新的参数数值=8), 则跳转到 CINCKEY
; 否则若新的参数数值<8, 显示新参数值
INC R1          ; R1 加 1
MOV A, @R1      ; 将地址 R0+2*R7+1 处的值移动到累加器

CLR C           ; 清除进位标志
RLC A           ; 通过进位左旋转, 低位塞 0, 0 对应端口灯亮
MOV P2, A       ; 将累加器移动到 P2
MOV @R1, A      ; 将累加器移动到地址 R1

CINCKEY:        ; 更新 OLDINC
MOV C, NEWINC
MOV OLDINC, C

; 然后检测参数数值减少键
DECKEY:
JB NEWDEC, CDECKEY ; 如果 NEWDEC 位被设置, 则跳转到 CDECKEY
JNB OLDDEC, CDECKEY ; 如果 OLDDEC 位未被设置, 则跳转到 CDECKEY
MOV A, R0
ADD A, R7
ADD A, R7

MOV R1, A       ; R1 = R0 + 2*R7
; MOV A, @R1    ; A = (R0 + 2*R7)
; DEC A         ; A = (R0 + 2*R7) - 1
; ANL A, #0FH   ; A = { (R0 + 2*R7) - 1 } & 0F
; MOV @R1, A    ;

INC R1          ; R1 = R0 + 2*R7 + 1
MOV A, @R1      ; A = (R0 + 2*R7 + 1)

SETB C
RRC A           ; 通过进位右旋转, 高位塞进 1, 1 对应端口灯灭
; CLR C
MOV P2, A
MOV @R1, A
CDECKEY:        ; 更新 OLDDEC
MOV C, NEWDEC
MOV OLDDEC, C

JMP STARTKEY

```

```

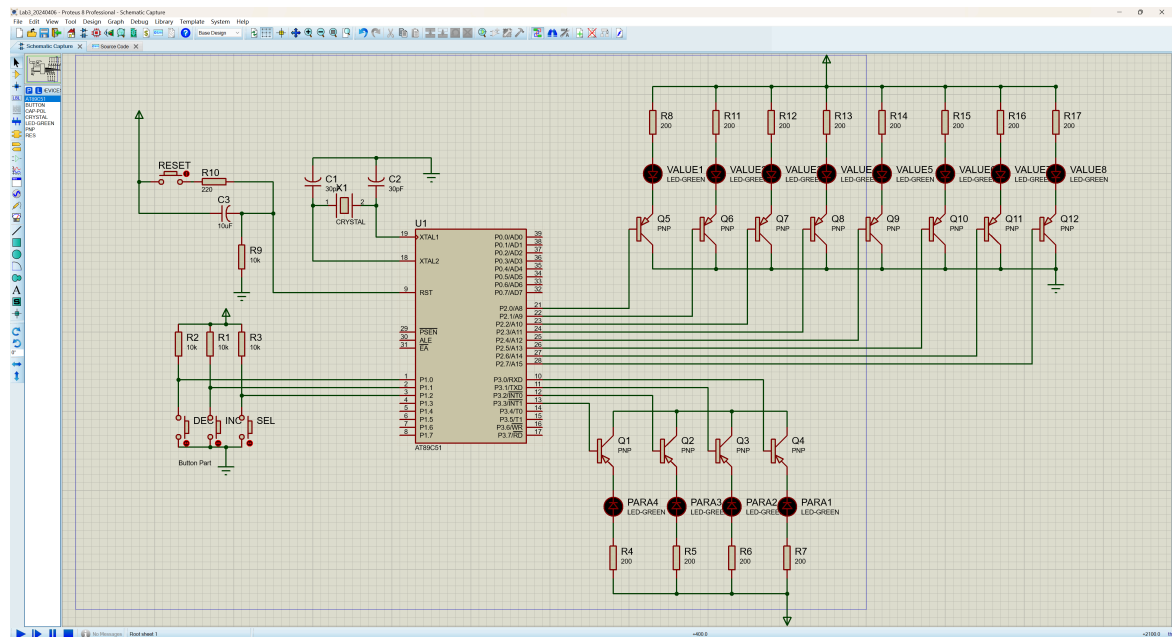
; =====
END

```

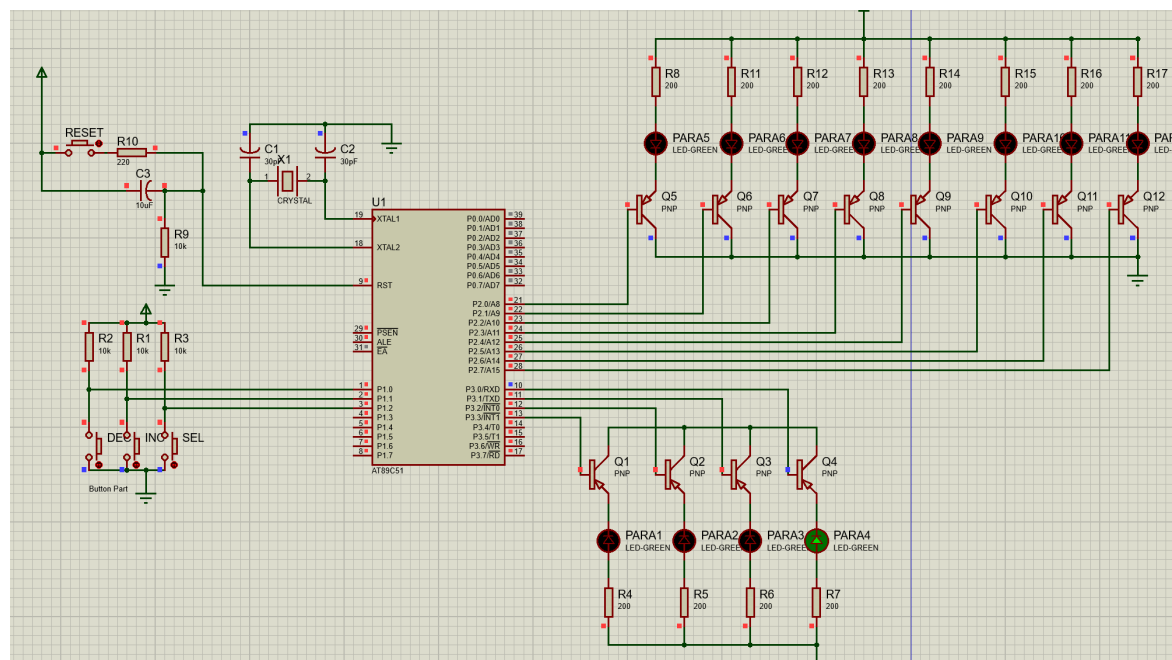

5. 实验结果和分析

5.1. 系统测试

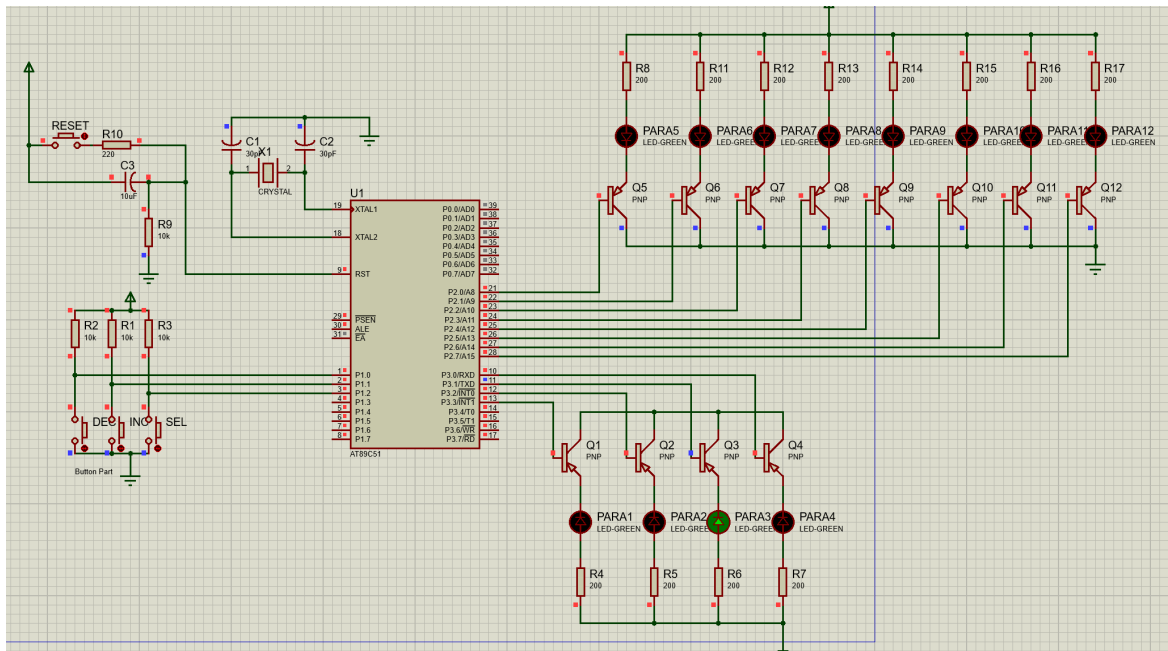
- 硬件电路设计图



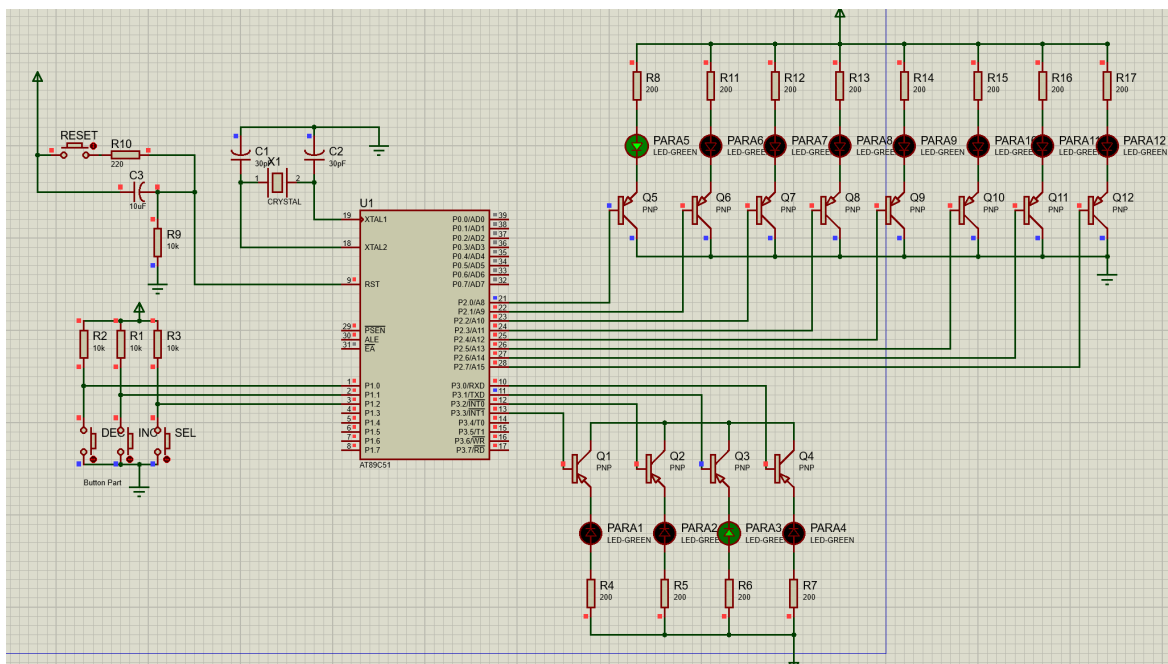
- 初始状态 最右侧参数灯亮，所有数据灯灭



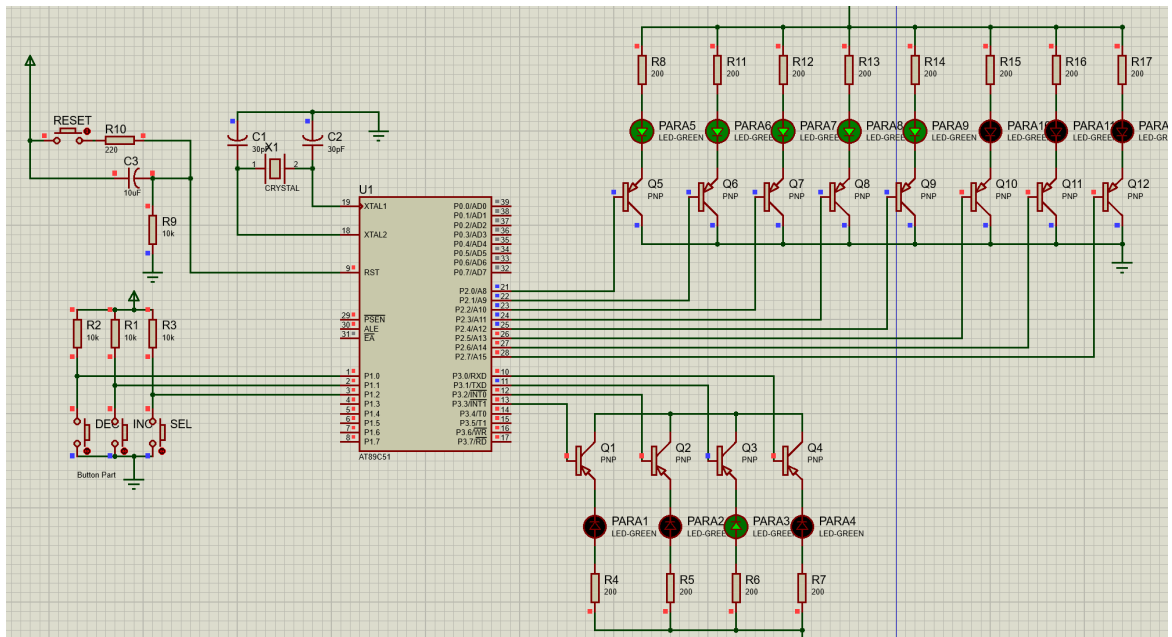
- 按下 SEL 切换参数 仅状态灯右 2 亮



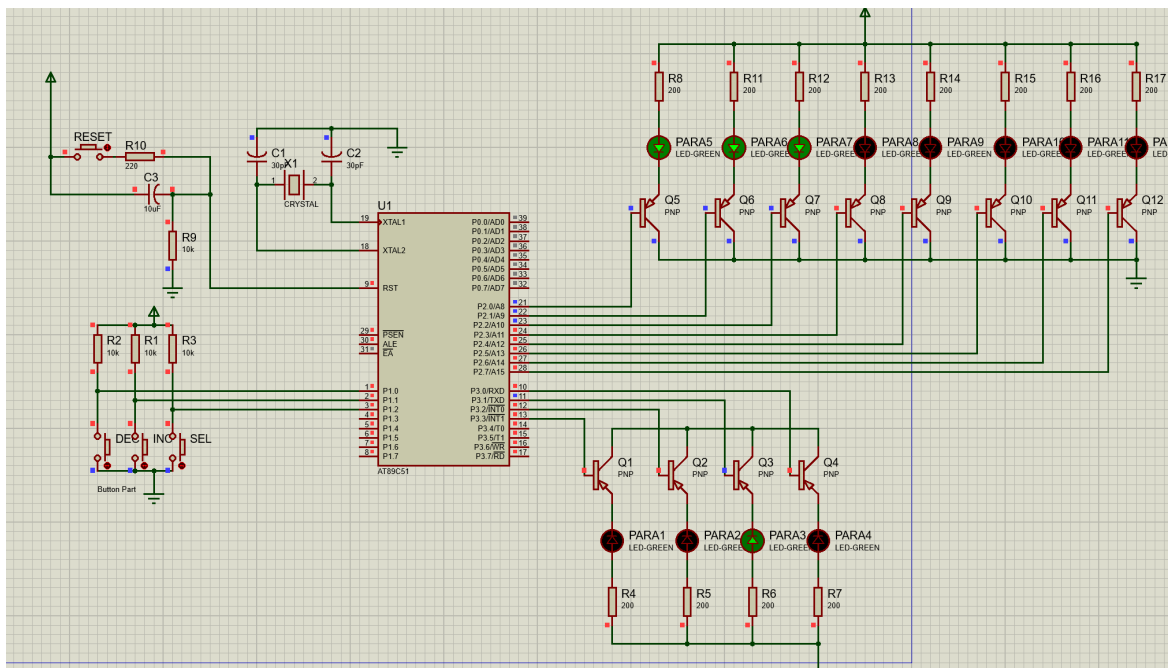
- 按下 INC 增加数值 参数右 2 灯亮时，数据左 1 灯亮，表示数据为 1



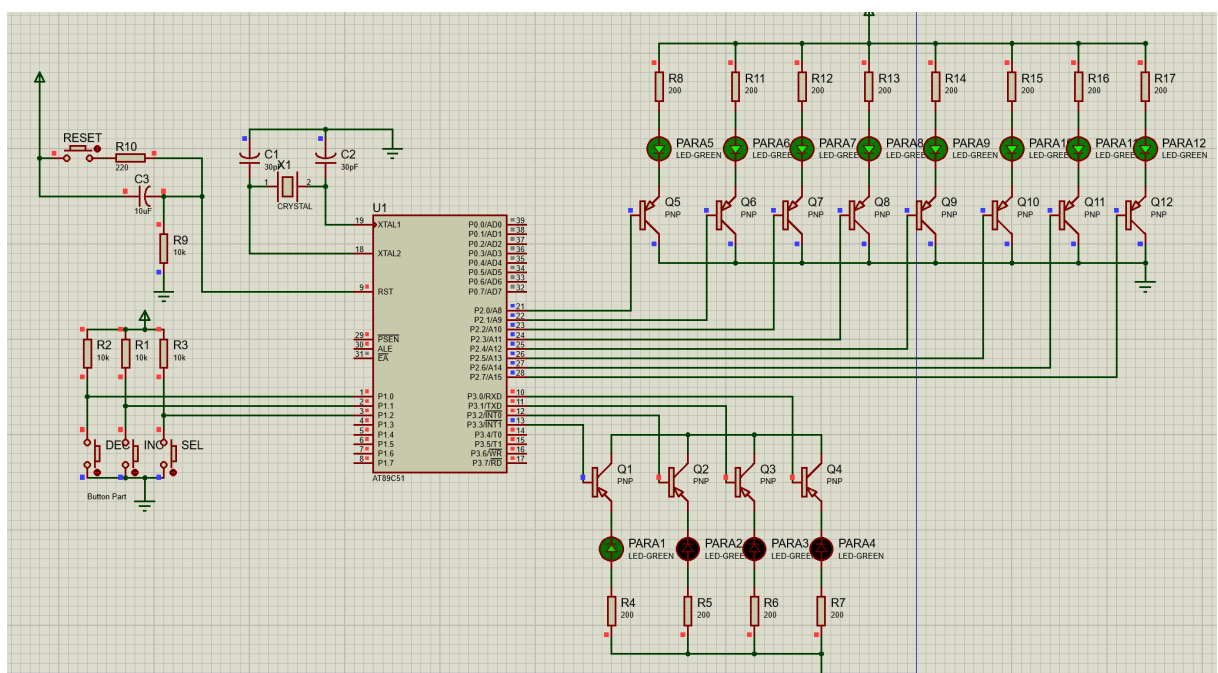
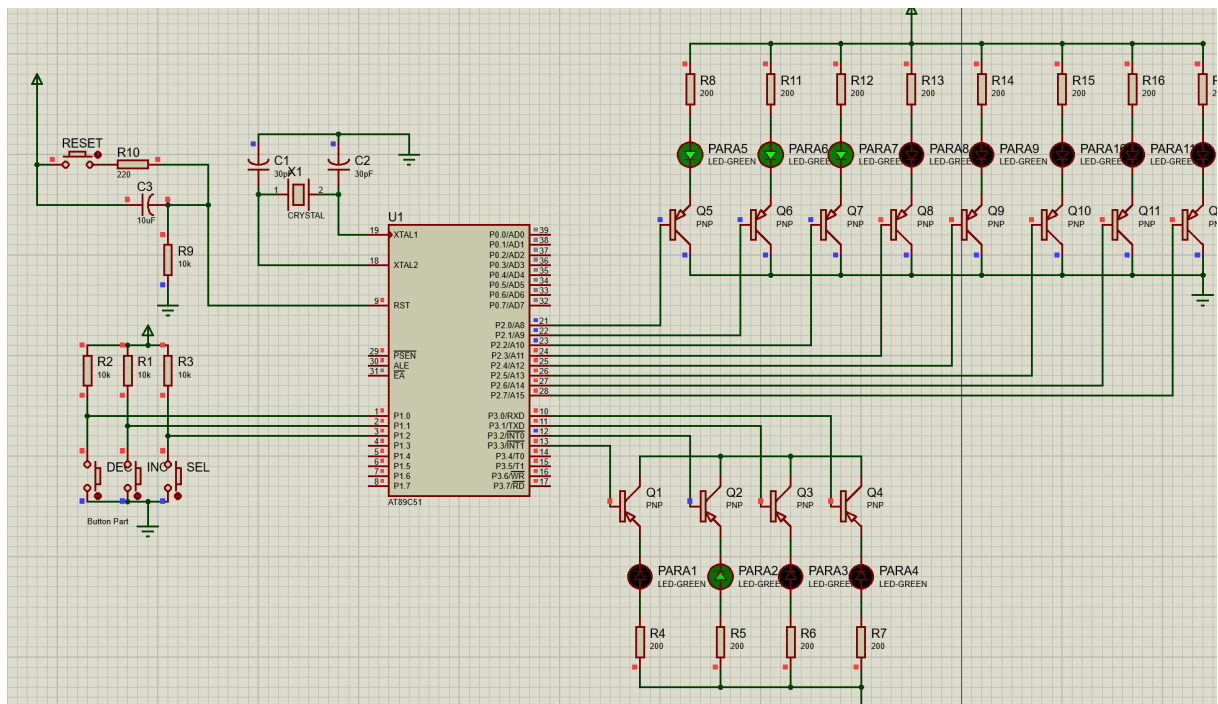
- 总共按下 5 次 INC 后，从左到右亮 5 个数据灯

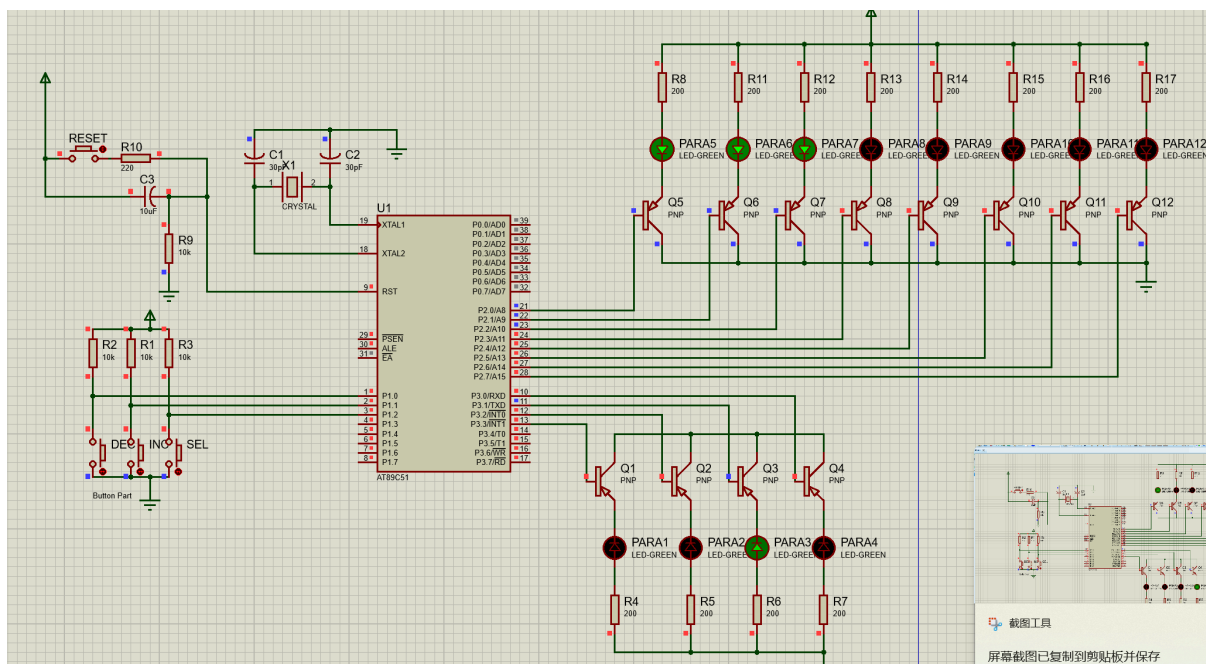
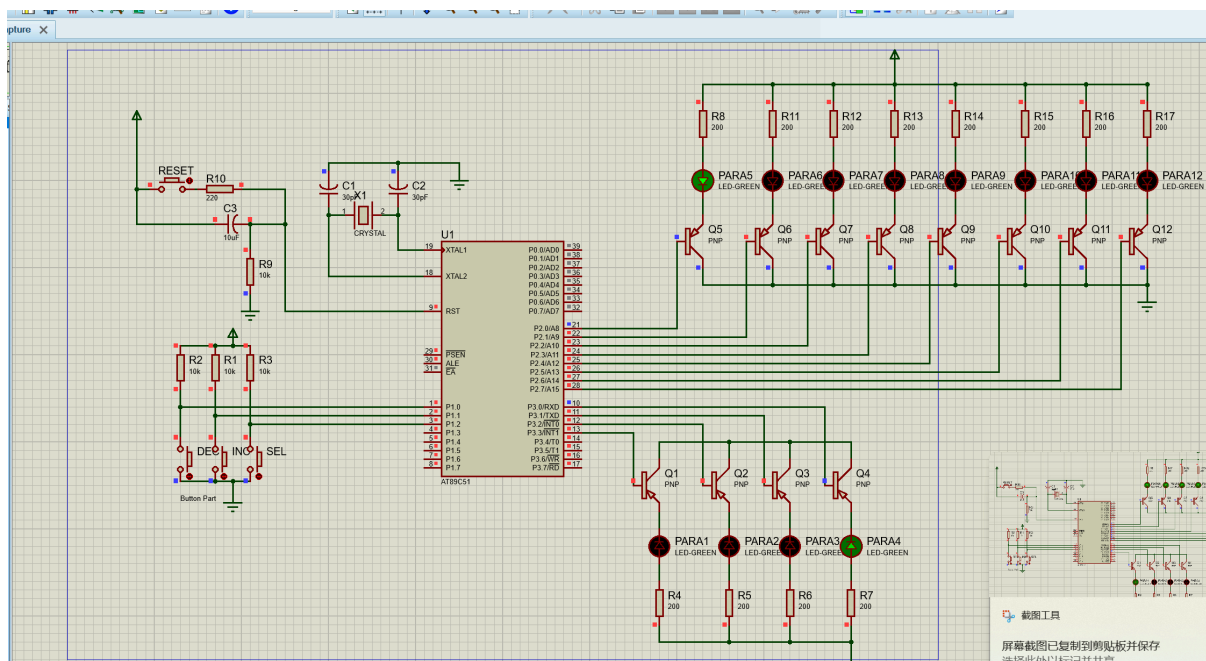


- 按下 2 次 DEC，数据减少 2，数据灯变为 3 个



- 切换到其余参数，经过一番把玩，赋予一些数据





- 且再次回到原来赋值的参数，其数据仍然保持不变

5.2. 分析

系统运行结果表明，本实验设计的基于 51 单片机的硬件电路和软件组成的系统，采用 LED, IO 硬件和循环、条件程序，实现了

- 对多参数输入，可用一个按键进行参数的选择，每按一次，更改一次参数，循环选择；
- 当按另一个按键时，对参数进行增量操作，直到最大值后，保持在最大值；
- 第三只按键每按一次执行减量操作，直到最小值，并保持在最小值。

完成了实验任务。

6. 思考和讨论

6.1. 扩展思考题

1. 你注意到什么电子设备上具有 2~3 按键参数设置功能？试分析工作原理。严格来说我最近没见到只有 2~3 按钮的设备，但类似的有洗衣机、电磁炉。例如洗衣机，有模式、水位调整，而模式、水位参数可以调节增大和减小。其工作原理一个应该类似本实验。即 MCU 接受按钮信号，当参数按钮按下时，切换参数灯，显示当前参数的数值；当增大或减小按钮，则修改存储器中对应值，并数码管显示对应数字。
2. 只有 3 个按键进行参数设置，你认为还有什么不方便的地方？试提出修改意见。参数只能循环选择，如果比较多就很烦，比如我寝室里的洗衣机有 10 个模式，错过了就得按 9 次。可以多加一些按钮，比如十个甚至九个。也可以加到丧心病狂的 26 个，甚至 88 个，这样就可以演奏音乐了。

6.2. DEBUG

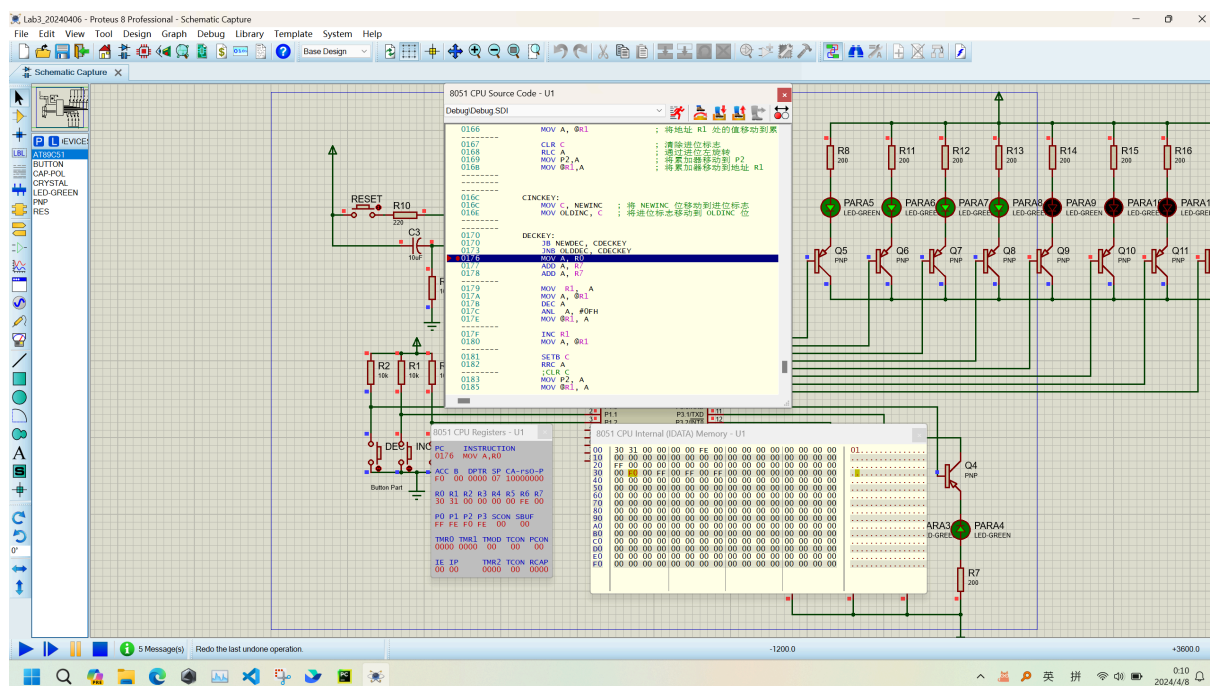
验收前，程序仿真时，如果某个参数 DEC 减小，则之后不能增大。我偶然发现删去

SETB C

RRC A ; 通过进位右旋转,高位塞进 1, 1 对应端口灯灭
;CLR C

CLR C 可以修复该 BUG，在验收时该功能正常。但回寝室后，就不能。

于是我调试程序



发现以下代码意义不明

```
MOV R1, A ; R1 = R0 + 2*R7
;MOV A, @R1 ; A = (R0 + 2*R7)
;DEC A ; A = (R0 + 2*R7) - 1
;ANL A, #0FH ; A = { (R0 + 2*R7) - 1 } & 0F
;MOV @R1, A ;
```


通过调试打断点、看内存和寄存器，那个 BUG，是在 DEC 之后再次 INCKEY 时，由于其偶数位地址数据存储位被置为 0x0E

```
;MOV A, @R1      ; 将地址 R0+2*R7 处的值移动到累加器
;INC A           ; 累加器加 1
;ANL A, #08H     ; 与 0x08 进行按位与
;JNZ CINCKEY     ; 如果结果为 1 (新的参数数值=8), 则跳转到 CINCKEY
; 否则若新的参数数值<8, 显示新参数值
```

导致 JNZ CINCKEY 违背我意愿地执行

这些代码似乎多余，删去也不影响运行，于是我将其注释掉

反正是很痛苦的调试，这软件太古早了

6.3. 总结

本实验花了我诸多时间。如果仅仅是能跑，那么把参考文档的代码小改一下就可以。

但我一开始没跑起来，因为 Word 文档的代码里少个分号;导致报错。

于是我研读文档，看了 3 个小时。

没理解。

我带着诸多困惑，第二天实验课去验收了。复制粘贴几行代码，然后就能过验收了。

当然还有一些 BUG，不过验收没有要求。

回到寝室，补了睡，然后开始写实验报告。于是我又把我的缝缝补补的代码掏出来看。

发现有些逻辑我看不懂，而且确实没必要似乎。我将稍后讨论。

终于，我似乎逐渐理解起来了。

首先，我强迫自己去分析到底程序调用了哪些硬件资源。

这是我前一天未解的困惑。因为参考文档写的不够清楚，用词模糊且没有解释。我求助同学、读代码，终于写出了前文的硬件资源部分。毕竟，如果不知道代码里的寄存器、内存这些硬件资源逻辑上是什么功能，那么代码阐释的他们的相互关系就更玄乎。

然后，我开始绘制流程图。

流程图能够跳出单行代码的局限，以功能性的、宏观的角度理解程序。是的，我又要吐槽参考文档的流程图了。不用自吹，我画的流程图肯定比它的好理解。由于理解代码和绘图不熟练，这一步花了我好几个小时。

但，几小时过后，我能说我懂这个程序了。

除了至今也不知道它的 30H, 32H 这四个偶数地址内存在干什么，我修改程序不用它们也能跑得好好的。

我觉得以上两部是精华所在。

6.4. 参考文档的思考

1. R3~R0

用 R3~R0 记录 4 个参数的值。

我一开始理解为分别存储四个参数的数值。

而实际上似乎是 R0 记录参数，R1 作临时变量，R2R3 没用到。参数数值存在内存 31H 33H 35H 37H

1. R6

文档没有说明 R6 是干什么的，我理解其作为参数 P3 口的信号数据

1. 参数存储资源分配表

参数	值域地址	初值	状态地址	初值
1	30H	00H	31H	FFH
2	32H	00H	33H	FFH
3	34H	00H	35H	FFH
4	36H	00H	37H	FFH

我一直没明白“值域”和“状态”是什么。我修改后的仅有程序参数数值存在内存 31H 33H 35H 37H。

1. 程序中的代码

在我的代码中被注释掉的部分